

Noct Finance — Implementation Architecture

Consolidated human-readable snapshot of the Markdown implementation package

01-NOCT_IMPLEMENTATION_ARCHITECTURE

Noct Finance — Implementation Architecture

Version: 1.0-architecture-freeze

Status: Implementation specification baseline

Primary audience: Human engineering team + implementation coding agent

Canonical format: Markdown

Scope: Noct V1 testnet private lending architecture using Vela, Horizon, UltraHonk and zkVerify.

0. How the implementation agent must use this document

This document is the architectural source of truth for the current Noct implementation baseline.

Normative language

- **MUST** = mandatory.
- **MUST NOT** = prohibited.
- **SHOULD** = recommended unless an explicit architecture decision overrides it.
- **MAY** = optional.
- **CONDITIONAL** = depends on benchmark/evidence.
- **OPEN** = not yet frozen; do not silently invent protocol semantics.

The implementation agent **MUST NOT** convert an old brainstorming idea into protocol behavior. Where a section is marked **OPEN**, implementation must stop at the interface/placeholder boundary and request or use a later architecture decision.

1. Product objective

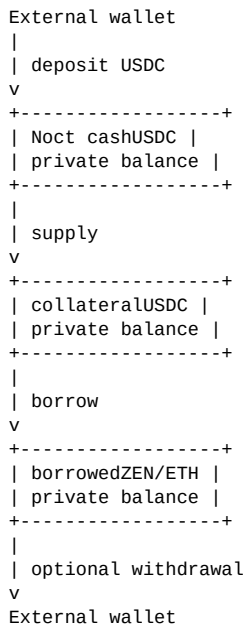
Noct is a privacy-preserving DeFi lending application intended to provide Aave-like lending primitives while preventing an external observer from associating a specific wallet with a specific private lending position.

The V1 testnet scope is intentionally narrow:

- **Supply/collateral asset:** USDC only.
- **Borrowable assets:** ZEN and ETH.
- **Protocol-funded liquidity:** Noct funds the ZEN and ETH borrow pools for the test application.
- **User position privacy:** collateral, debt, LTV, health factor, balances and transition details are private.
- **Aggregate protocol information:** may be public where it does not reveal an individual position.
- **ZK:** used where it materially strengthens private state-transition authorization, subject to performance gates.
- **Vela:** used as the confidential execution/state platform.
- **Horizon:** settlement/on-chain coordination layer for the deployment target.

2. Core mental model

A user's assets have distinct lifecycle states.



The important rule is:

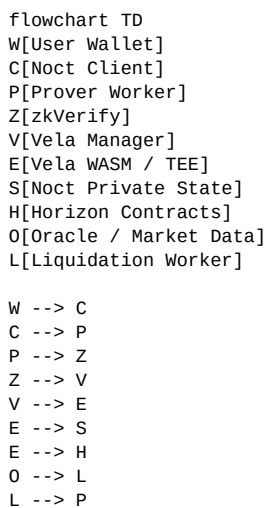
> **Deposit is not supply.**

A deposited USDC balance may remain as private cash and may be withdrawn without ever becoming collateral.

Likewise, when collateral is released, it returns to the user's private cash balance first. The user then decides whether to withdraw that cash externally.

Borrowed assets similarly enter a private borrowed-asset balance first; the user may then withdraw them to an external wallet.

3. System boundaries



Responsibility boundary

| Component | Responsibility |

---|---

| User wallet | User authorization/signing and external asset ownership |

| Noct client | UI, transaction construction, private-state interaction, status |

| Prover | Generate ZK proofs from authorized private witnesses |

| zkVerify | Verify supported ZK proofs and optionally aggregate them |

| Vela Manager | Request orchestration, blockchain interaction, state persistence/coordination |

| Vela WASM/TEE | Confidential application execution |

| Noct private state | Account/position state that must not be exposed to observers |

| Horizon contracts | On-chain custody/coordination/settlement boundary |

| Oracle/market data | Prices and risk inputs |

| Liquidation worker | Detect eligible positions and drive liquidation transitions |

4. On-chain vs private/off-chain data

4.1 Private

The following should remain inside the confidential Noct state or be represented only through commitments/proofs:

- account private identity linkage;
- cash USDC balance;
- supplied USDC/collateral amount;
- ZEN debt;
- ETH debt;
- borrowed-but-not-yet-withdrawn balances;
- LTV;
- health factor;
- user-specific interest/debt accounting;
- user-specific transition history;
- pending private state data;
- private nonces/nullifiers as applicable.

4.2 Public / observable

Only data required for coordination, settlement, configuration and protocol-level observability should be exposed:

- application identity;
- protocol configuration commitments;
- asset configuration identifiers;
- state commitments/roots;
- transition commitments;
- proof/verification references;
- protocol aggregate metrics where privacy is preserved;
- public contract events required for reconciliation;
- external settlement transactions that inherently occur on a public chain.

Privacy principle

An observer should not be able to take a public transaction and reliably conclude:

> “This exact wallet owns this exact Noct lending position with this collateral, debt and health factor.”

Noct therefore does not promise that every external blockchain interaction is invisible. The V1 privacy boundary is primarily **private lending state and private state transitions**.

5. V1 asset model

| Asset | Deposit | Supply/collateral | Borrow | Protocol liquidity |

|---|---|---|---|---|

| USDC | YES | YES | NO | User supplied |

| ZEN | NO | NO | YES | Noct-funded |

| ETH | NO | NO | YES | Noct-funded |

V1 intentionally avoids a multi-collateral/multi-supply model.

This reduces:

- oracle complexity;
- risk-parameter complexity;
- circuit complexity;
- state-transition complexity;
- liquidation complexity;
- testing surface.

6. Account lifecycle

A simplified account lifecycle is:

```
UNINITIALIZED
|
| account initialization
v
ACTIVE
|
+--> CASH ONLY
|
+--> COLLATERALIZED
|
+--> BORROWED
|
+--> BORROWED + CASH
|
+--> BORROWED + PENDING WITHDRAWAL
|
+--> LIQUIDATION ELIGIBLE
|
+--> LIQUIDATION
|
+--> ACTIVE
```

These are conceptual states, not necessarily enum values. The canonical state fields and transition rules are specified in 02-NOCT_STATE_MACHINE.md.

7. ZK architecture

Noct does not hard-code a single proof-consumption strategy into every transition.

```
flowchart TD
    T[Noct Transition]
    I[ZK Verification Interface]
    D[Direct / verify-only]
    A[zkVerify aggregation receipt]
    X[Transition Authorization]
    V[Vela]
    T --> I
    I --> D
    I --> A
    D --> X
    A --> X
    X --> V
```

Frozen decision

Production target: direct consumption of zkVerify aggregation receipts by the Noct/Horizon on-chain layer.

Condition: this target is not accepted into the user-facing critical path until end-to-end performance is benchmarked.

Fallback: verify-only/app-side consumption may be used for testnet or a transition if aggregation latency/complexity fails the performance gate.

zkVerify currently documents Noir UltraHonk support with V0_84, V3_0 and Legacy submission variants, with limits including 32 public inputs and evaluation-domain size 2^{25} . See the current zkVerify supported-proofs documentation.

8. Vela architecture

The current Vela repository describes a privacy-preserving execution platform based on AWS Nitro Enclaves, encrypted state management and blockchain coordination. The Vela runtime executes WASM; the Manager coordinates blockchain requests/state; the storage layer supports versioned application state.

Noct should treat Vela as the confidential execution substrate, not as the lending protocol itself.

```
flowchart LR
    BC[Horizon / ProcessorEndpoint]
    M[Vela Manager]
    E[Nitro Enclave Executor]
    W[Noct WASM]
    DB[Versioned private state]
    BC --> M
    M <--> E
    E --> W
    M --> DB
```

The Vela Starter Kit provides the local development stack and a WASM application development path. Vela Nova is a concrete private-transfer application demonstrating private balances, transfers and withdrawals. These repositories are implementation references, not permission to copy their business logic into Noct.

9. Vela security boundary

The Noct WASM application MUST treat its private state as authoritative only when:

1. the request is correctly authenticated;
2. the request is bound to the correct Noct application;
3. the state version/root is valid;
4. transition preconditions pass;
5. the proof/authorization required by that transition passes;
6. replay protection passes;
7. the resulting state transition is deterministic.

The implementation must preserve Vela's state versioning/rollback properties and must not introduce non-deterministic state changes.

10. Performance is a hard requirement

No ZK component may enter a user-facing critical path without benchmarking.

The full pipeline to benchmark is:

```
witness preparation
↓
UltraHonk proving
↓
zkVerify submission
↓
```

```
zkVerify verification
↓
aggregation (if used)
↓
receipt availability
↓
Horizon/Noct receipt consumption
↓
Vela execution
↓
final settlement
```

Required metrics:

- p50;
- p95;
- p99;
- maximum;
- CPU;
- RAM;
- proof size;
- circuit constraints;
- failure/retry rate.

Browser proving is not mandatory for V1. An external prover-worker architecture **MUST** be supported as the baseline implementation option.

11. Liquidation architecture

Liquidation must not depend on an end-user browser.

```
flowchart TD
    M[Market / Oracle Data]
    D[Liquidation Detector]
    P[Dedicated Prover Worker]
    Z[zkVerify]
    N[Noct Liquidation Transition]
    V[Vela]
    M --> D
    D --> P
    P --> Z
    Z --> N
    N --> V
```

Liquidation performance is a separate benchmark class because delayed liquidation can directly affect protocol solvency.

Exact liquidation formulas, auction/repayment mechanics and liquidator incentive parameters must remain aligned with the finalized state-machine specification.

12. Security principles

Noct **MUST** prioritize:

1. privacy correctness;
2. state-transition correctness;
3. replay protection;
4. authorization correctness;
5. deterministic execution;
6. failure recovery;

7. asset conservation;
8. liquidation safety;
9. proof verification correctness;
10. performance.

Speed must not be achieved by weakening a security invariant.

13. Implementation constraints

The implementation agent MUST NOT:

- expose private balances in public events;
- expose user LTV/health factor publicly;
- infer account identity from state commitments;
- allow deposit to automatically imply supply;
- allow collateral release to bypass the private cash balance;
- treat borrowed balance and debt as the same state variable;
- decrement debt when borrowed assets are withdrawn;
- allow a pending withdrawal to be spent again;
- accept a stale state root;
- accept a replayed transition;
- depend on browser proving for liquidation;
- assume zkVerify aggregation latency without benchmark data;
- silently add assets to V1.

14. Source references

Implementation should use the current repositories as primary Vela references:

- Vela Starter Kit: <https://github.com/HorizenOfficial/vela-starterkit>
- Vela Nova: <https://github.com/HorizenOfficial/vela-nova>
- Vela core: <https://github.com/HorizenOfficial/vela>
- Vela Facilitator: <https://github.com/HorizenOfficial/vela-facilitator>
- zkVerify supported proofs: https://docs.zkverify.io/architecture/supported_proofs
- zkVerify UltraHonk verifier: https://docs.zkverify.io/architecture/verification_pallets/ultrahonk
- zkVerifyJS/aggregation: <https://docs.zkverify.io/overview/zkverifyjs>

15. Document dependencies

- 02-NOCT_STATE_MACHINE.md is authoritative for state and transitions.
- 03-NOCT_ZK_SPECIFICATION.md is authoritative for ZK.
- 04-NOCT_VELA_INTEGRATION.md is authoritative for Vela integration.
- 05-NOCT_IMPLEMENTATION_PLAN.md is authoritative for build sequencing.

If documents conflict, implementation must stop and the conflict must be resolved explicitly.

02-NOCT_STATE_MACHINE

Noct Finance — State Machine Specification

Version: 1.0-architecture-freeze

Status: Frozen baseline for implemented decisions

Purpose: Define the private account model and state-transition semantics.

1. Core rule

Noct separates:

1. **cash** — deposited but not supplied;
2. **collateral** — supplied USDC;
3. **borrowed balance** — debt asset received from the protocol but not yet externally withdrawn;
4. **debt** — outstanding liability.

These are distinct concepts and MUST NOT be collapsed into one balance.

2. V1 asset constraints

```
SUPPLY_ASSET = USDC
```

```
BORROW_ASSETS = {  
  ZEN,  
  ETH  
}
```

V1 users deposit USDC only.

ZEN and ETH are borrow-only assets for the V1 lending market and are funded by Noct.

3. Conceptual account state

```
AccountState {  
  identityCommitment  
  accountNonce  
  
  cashUSDC  
  
  collateralUSDC  
  
  borrowedZEN  
  borrowedETH  
  
  debtZEN  
  debtETH  
  
  protocolStateVersion  
  stateCommitment  
  transitionNonce  
  
  pendingWithdrawal  
  pendingSettlement  
  
  status  
}
```

This is a conceptual schema. Exact field encodings, integer widths, hashes and serialization formats must be fixed in implementation after the circuit/public-input design is finalized.

4. Conservation relationships

USDC

```
private user USDC =  
cashUSDC  
+ collateralUSDC  
+ any explicitly pending USDC state
```

A user can deposit repeatedly:

```
Day 1: deposit 10 USDC  
cashUSDC = 10
```

```
Day 2: deposit 5 USDC  
cashUSDC = 15
```

The system MUST support repeated deposits.

5. Deposit transition

DEPOSIT_USDC

Purpose: move USDC from the external custody boundary into the user's private Noct cash balance.

Preconditions

- asset is USDC;
- amount > 0;
- external deposit is valid;
- request is bound to the correct Noct application;
- replay protection passes.

State change

```
cashUSDC' = cashUSDC + amount
```

No collateral is created.

No debt is created.

No supply is implied.

Example

```
Before:  
cashUSDC = 0  
collateralUSDC = 0
```

```
Deposit 10 USDC
```

```
After:  
cashUSDC = 10  
collateralUSDC = 0
```

6. Supply transition

SUPPLY_USDC

Purpose: move existing private cash USDC into collateral.

Preconditions

```
amount > 0  
amount <= cashUSDC
```

State change

```
cashUSDC' = cashUSDC - amount
collateralUSDC' = collateralUSDC + amount
```

Example

```
Before:
cashUSDC = 15
collateralUSDC = 0
```

Supply 13

```
After:
cashUSDC = 2
collateralUSDC = 13
```

The user does not need to supply immediately after depositing.

7. Borrow transition

BORROW_ZEN / BORROW_ETH

Purpose: create a debt against eligible USDC collateral and credit the borrowed asset to the user's private borrowed balance.

Core rule

The borrowed asset first appears inside Noct as:

```
borrowedZEN
```

or:

```
borrowedETH
```

It is not automatically sent to the user's external wallet.

Preconditions

The transition MUST verify:

- user authorization;
- valid private state;
- sufficient collateral;
- asset borrowability;
- protocol liquidity;
- risk parameters;
- resulting LTV/health constraints;
- transition nonce/replay protection;
- correct state root/version;
- required ZK proof/verification.

State change

For ZEN:

```
borrowedZEN' = borrowedZEN + amount
debtZEN' = debtZEN + debtAmount
```

For ETH:

```
borrowedETH' = borrowedETH + amount
debtETH' = debtETH + debtAmount
```

The exact pricing, interest-rate and debt-accounting formulas must be frozen in the risk-parameter section before production deployment.

8. Example borrow

Alice:

```
cashUSDC = 0
collateralUSDC = 100
debtZEN = 0
borrowedZEN = 0
```

She borrows 20 ZEN worth \$50.

After:

```
cashUSDC = 0
collateralUSDC = 100
debtZEN = $50-equivalent
borrowedZEN = 20 ZEN
```

Alice can keep the 20 ZEN inside Noct or request withdrawal.

9. Withdraw borrowed asset

WITHDRAW_BORROWED_ASSET

Purpose: move a previously borrowed asset from the private borrowed balance to the user's external wallet.

Required state model

The borrowed balance is locked before external settlement.

```
borrowedZEN = 20
lockedZEN = 0
```

Request 20 ZEN withdrawal

```
borrowedZEN = 20
lockedZEN = 20
```

Successful settlement

```
borrowedZEN = 0
lockedZEN = 0
debtZEN = unchanged
```

Failed settlement

```
borrowedZEN = 20
lockedZEN = 0
debtZEN = unchanged
```

Security invariant

A pending withdrawal **MUST** prevent the same borrowed balance from being withdrawn twice.

10. Collateral release

When Alice releases collateral:

```
collateralUSDC -= amount
cashUSDC += amount
```

The released collateral does **not** immediately go to the external wallet.

Example:

```
Before:
cashUSDC = 5
collateralUSDC = 100
debtZEN = $50
```

Release 20 USDC

After:

```
cashUSDC = 25
collateralUSDC = 80
debtZEN = $50
```

Alice can then choose:

```
cashUSDC -> external wallet
```

This preserves the separation between protocol cash and external withdrawal.

11. Withdraw cash USDC

WITHDRAW_CASH_USDC

Purpose: withdraw USDC that is already in private cash.

Preconditions

```
amount > 0
amount <= cashUSDC
```

State behavior

The amount is locked for settlement.

On success:

```
cashUSDC -= amount
lockedUSDC -> 0
```

On failure:

```
cashUSDC unchanged
lockedUSDC -> 0
```

This transition does not create or reduce debt.

12. Repayment

Repayment must reduce outstanding debt; it is distinct from withdrawing or moving a borrowed balance.

Conceptually:

```
debtZEN' = debtZEN - repayment
```

or:

```
debtETH' = debtETH - repayment
```

The exact handling of:

- overpayment;
- repayment larger than debt;
- accrued interest;
- rounding;
- partial repayment;
- repayment from private cash versus externally supplied repayment;

must be implemented only according to the finalized repayment specification.

Do not invent an overpayment policy in code.

13. Liquidation

Liquidation is a protocol transition, not a user-wallet operation.

The liquidation engine must establish that the private position is liquidatable without publicly exposing the user's position.

Conceptual flow:

```
Private position
|
| ZK/risk proof
v
Liquidation eligible
|
v
Liquidation execution
|
+--> debt reduced
+--> collateral reduced
+--> liquidator settlement
|
v
New private state
```

Exact liquidation mechanism, liquidation bonus, close factor, oracle assumptions and liquidator settlement fields are **OPEN until the liquidation transition specification is explicitly frozen**.

14. Global transition invariants

Every transition MUST preserve:

I1 — No negative balances

```
all asset balances >= 0
```

I2 — No debt creation without collateral/risk authorization

I3 — No debt reduction without a valid repayment/liquidation event

I4 — No external withdrawal without an internal balance

I5 — No double withdrawal

I6 — No stale-state transition

I7 — No replayed transition

I8 — Correct asset binding

I9 — Correct application binding

I10 — Deterministic state transition

I11 — Private position fields are not emitted publicly

I12 — Failed external settlement restores spendable private balance

15. State commitment

The implementation should expose only a commitment/root representing the private state.

A conceptual commitment is:

```
stateCommitment =
Hash(
identityCommitment,
cashUSDC,
collateralUSDC,
borrowedZEN,
borrowedETH,
debtZEN,
debtETH,
```

```
accountNonce,  
pendingState,  
protocolStateVersion  
)
```

The exact hash/circuit encoding is OPEN until the ZK specification freezes it.

16. Transition lifecycle

Every transition follows the same conceptual pipeline:

```
flowchart TD  
A[Current private state] --> B[Build transition intent]  
B --> C[Generate witness]  
C --> D[Generate proof]  
D --> E[Verify proof]  
E --> F[Execute deterministic transition]  
F --> G[Compute new private state]  
G --> H[Commit new state]  
H --> I[External settlement if required]
```

A failed transition MUST NOT partially mutate the authoritative private state.

17. Freeze rule

The state machine is frozen only for decisions explicitly marked above.

The implementation agent MUST NOT silently fill OPEN risk or liquidation semantics.

03-NOCT_ZK_SPECIFICATION

Noct Finance — ZK Specification

Version: 1.0-architecture-freeze

Status: ZK architecture baseline; circuit details evolve through benchmark gates.

1. Objective

ZK proves that a private Noct state transition is valid without revealing the user's complete lending position.

The proof system must establish statements such as:

- the user controls the relevant private state;
- the current state is valid;
- the transition follows protocol rules;
- collateral/risk constraints hold;
- the resulting state is correctly derived;
- the transition is not a replay.

The proof **MUST NOT** expose private collateral, debt or health-factor values merely because the transition is verified.

2. ZK pipeline

```
flowchart LR
  S[Private State] --> W[Witness Builder]
  W --> P[UltraHonk Prover]
  P --> Z[zKVerify]
  Z --> R[Verification mode]
  R --> D[Direct / verify-only]
  D --> A[Aggregation]
  A --> AR[Aggregation Receipt]
  AR --> X[Transition Authorization]
```

3. UltraHonk compatibility

Current zkVerify documentation lists Noir UltraHonk support for:

- Noir $\geq$ v1.0.0-beta.14;
- bb $\geq$ v3.0.0;
- bb.js $\geq$ v3.0.0;
- Keccak256;
- submission variants V0_84, V3_0, and Legacy.

Current documented limits include:

- maximum public inputs: 32;
- maximum evaluation domain size: 2^{25} .

The implementation **MUST** pin and test exact toolchain versions rather than using floating versions.

4. Public/private input design

Private witness candidates

Examples:

- private account secret/key material;
- current private balances;
- private debt;
- private collateral;
- private nonce;
- Merkle/state membership witnesses;
- transition-specific secret randomness.

Public input candidates

Examples:

- state commitment/root;
- protocol configuration commitment;
- transition identifier;
- asset identifier;
- amount commitment or authorized public amount where required;
- nonce/nullifier commitment;
- previous/new state commitment;
- verification-domain identifiers.

The final public-input list MUST be frozen per circuit before deployment.

5. Privacy requirement

The circuit must avoid making these directly public:

```
Alice.collateralUSDC
Alice.debtZEN
Alice.debtETH
Alice.healthFactor
Alice.LTV
```

Instead, the proof establishes that the hidden values satisfy the required inequalities.

Example concept:

```
private:
collateral = 100
debt = 50
oraclePrice = ...

public:
stateRoot
protocolConfigHash
transitionId
newStateCommitment

proof:
"The private state satisfies the borrow constraint."
```

6. Circuit decomposition

Do not build one monolithic lending circuit.

Recommended circuit families:

```
deposit validation
supply
borrow
repay
release collateral
withdraw cash
withdraw borrowed asset
liquidation
```

Shared gadgets/primitives MAY include:

- commitment hashing;
- state membership;
- nullifier/replay protection;
- signature/authorization verification;
- range checks;
- arithmetic constraints;
- risk inequalities.

7. Borrow proof

The borrow circuit should prove, without revealing the user's full position:

1. Current state commitment is valid.
2. User is authorized.
3. USDC collateral exists.
4. Requested borrow asset is enabled.
5. Protocol liquidity is sufficient.
6. Borrow amount satisfies risk constraints.
7. New debt is correctly derived.
8. New state commitment is correctly computed.
9. Transition nonce is valid.
10. The transition has not been replayed.

The exact collateral/risk formula must come from the finalized risk configuration.

8. Repay proof

The repay circuit should prove:

1. State is current.
2. Debt exists.
3. Repayment amount is valid.
4. New debt is correctly computed.
5. New state commitment is correct.
6. Replay protection passes.

Overpayment behavior remains an explicit protocol decision and must not be inferred by the implementation agent.

9. Withdrawal proof

For cash withdrawal:

```
cashUSDC >= withdrawalAmount
```

For borrowed-asset withdrawal:

```
borrowedAsset >= withdrawalAmount
```

The withdrawal proof must bind:

- account;
- asset;
- amount;

- transition nonce;
- current state;
- destination authorization;
- application identity.

10. Liquidation proof

Liquidation should prove that the hidden position satisfies the liquidation condition under the approved oracle/risk configuration.

The proof must avoid exposing the complete position.

Liquidation is expected to use dedicated infrastructure rather than browser proving.

11. Verification architecture

Noct uses an abstraction:

```
interface ZKVerifier {
  verifyTransition(proof, verificationContext) -> VerificationResult
}
```

Implementations:

```
DirectVerifier
AggregationReceiptVerifier
```

The state machine should depend on the interface, not the concrete mechanism.

12. zkVerify aggregation

zkVerify domains support aggregation. Current zkVerifyJS documentation describes:

- proof submission to a domain;
- aggregation after the domain's aggregation size is reached;
- NewAggregationReceipt;
- domain ID;
- aggregation ID;
- a Merkle-root receipt;
- retrieval of the statement path for the aggregate.

Noct therefore targets aggregation receipts for the production decentralized architecture.

However, aggregation latency is a performance-gated decision.

13. Performance gate

A ZK feature cannot be accepted merely because the proof verifies.

For every circuit measure:

```
Circuit constraints
Witness generation
Proof generation
Proof size
Local verification
zkVerify submission
zkVerify verification
Aggregation
Receipt publication/availability
```

Noct/Vela consumption
Horizon confirmation
Total end-to-end latency
CPU
RAM
Failure rate

Use:

p50
p95
p99
max

Do not report only averages.

14. Prover architecture

V1 MUST support an external prover-worker architecture.

```
flowchart TD
  C[Noct Client] --> Q[Private Proof Request]
  Q --> P[Dedicated Prover Worker]
  P --> U[UltraHonk]
  U --> Z[zKVerify]
  Z --> V[Vela / Noct]
```

The prover service must not become a plaintext store of user financial positions.

The design must define:

- encryption in transit;
- access control;
- retention policy;
- memory cleanup;
- logging policy;
- failure/retry behavior;
- witness confidentiality.

Browser proving can be supported experimentally but is not the V1 critical-path requirement.

15. Benchmark harness

The benchmark harness MUST be built before final ZK integration.

Test:

| Transition | Prove | Verify | Aggregate | Receipt | E2E |

|---|---|---|---|---|

| Supply | ✓ | ✓ | ✓ | ✓ | ✓ |

| Borrow | ✓ | ✓ | ✓ | ✓ | ✓ |

| Repay | ✓ | ✓ | ✓ | ✓ | ✓ |

| Release collateral | ✓ | ✓ | ✓ | ✓ | ✓ |

| Withdraw | ✓ | ✓ | ✓ | ✓ | ✓ |

| Liquidation | ✓ | ✓ | ✓ | ✓ | ✓ |

16. Circuit size budget

Because zkVerify documents limits for UltraHonk, circuit design must stay within the supported public-input and evaluation-domain constraints.

The implementation MUST fail the build/CI if a circuit exceeds the selected deployment profile.

17. Version pinning

The project MUST pin:

- Noir version;
- Barretenberg/bb version;
- bb.js version if used;
- circuit source hash;
- verification key hash;
- zkVerify submission variant;
- protocol configuration version.

A proof generated under an incompatible circuit/VK version must not be accepted as a current proof.

18. ZK failure handling

Possible failures:

```
witness generation failed
proof generation failed
proof invalid
zkVerify submission failed
verification rejected
aggregation unavailable
receipt unavailable
receipt mismatch
state root stale
Vela execution failed
Horizon settlement failed
```

Each failure must have deterministic handling and MUST NOT partially commit private state.

19. Source references

- zkVerify supported proofs: https://docs.zkverify.io/architecture/supported_proofs
- UltraHonk verifier: https://docs.zkverify.io/architecture/verification_pallets/ultrahonk
- zkVerifyJS aggregation: <https://docs.zkverify.io/overview/zkverifyjs>

04-NOCT_VELA_INTEGRATION

Noct Finance — Vela Integration Specification

Version: 1.0-architecture-freeze

Status: Vela integration baseline.

1. Purpose

Vela is the confidential execution substrate for Noct.

Noct supplies the lending state machine and business rules.

Vela supplies the confidential WASM execution environment, state coordination and blockchain-facing execution model.

The Noct implementation must be based primarily on the current Vela repositories rather than stale high-level documentation.

Primary references:

- [HorizenOfficial/vela-starterkit](#)
- [HorizenOfficial/vela-nova](#)
- [HorizenOfficial/vela](#)
- [HorizenOfficial/vela-facilitator](#)

2. Current Vela architecture relevant to Noct

The current Vela repository describes:

```
Manager
|
+-- blockchain interaction
+-- request fetching
+-- state persistence
+-- communication with Executor
|
Executor
|
+-- Wasmtime
+-- WASM execution
+-- encrypted state handling
+-- cryptographic keysets
+-- attestation payloads
```

The Executor runs inside an AWS Nitro Enclave in the intended Vela architecture.

3. Noct application inside Vela

Noct should be implemented as a Vela WASM application:

```
flowchart TD
    H[Horizon / ProcessorEndpoint]
    M[Vela Manager]
    E[Nitro Enclave Executor]
    N[Noct WASM]
    S[Noct Versioned Private State]

    H --> M
    M --> E
    E --> N
    N --> S
    S --> N
    N --> E
    E --> M
    M --> H
```

The Noct WASM application must be deterministic.

4. State persistence

Noct state should use the Vela application-state mechanism rather than inventing a second uncontrolled state database.

The current Vela codebase describes versioned LevelDB application state and rollback support.

Noct must maintain:

- current application state;
- state version;
- previous state/version where required for rollback;
- state commitment;
- transition metadata required for reconciliation.

5. Request lifecycle

Conceptual lifecycle:

1. Client constructs request.
2. Request is authenticated/encrypted as required by Vela.
3. Blockchain request is submitted.
4. Vela Manager fetches request.
5. Executor receives request.
6. Noct WASM validates/decrypts/processes it.
7. Noct state transition executes deterministically.
8. Result/AppEvent data is returned.
9. Manager persists the resulting state.
10. Blockchain settlement/update is performed.

The exact request encoding must follow the current Vela Starter Kit / Nova client patterns.

6. Noct transition adapter

Noct should expose a deterministic transition dispatcher:

```
process(request) -> result
```

Conceptual request types:

```
INIT_ACCOUNT
DEPOSIT_USDC
SUPPLY_USDC
BORROW_ZEN
BORROW_ETH
REPAY_ZEN
REPAY_ETH
RELEASE_COLLATERAL
WITHDRAW_CASH_USDC
WITHDRAW_BORROWED_ASSET
LIQUIDATE
```

These names are implementation-level labels and can be mapped to the actual Vela request model.

7. Private state rule

The WASM application is the authoritative execution environment for private lending state.

It must not emit:

```
Alice collateral = 100
Alice debt = 50
Alice HF = 1.2
```

in plaintext public events.

Instead it should emit only the minimum public information needed for:

- settlement;
- reconciliation;
- commitments;
- transition status;
- protocol aggregate reporting.

8. External settlement

Vela's current ecosystem includes an on-chain ProcessorEndpoint and private transfer patterns.

For Noct, external settlement should follow a lock/settle/unlock model.

Example:

```
borrowedZEN = 20
|
withdraw request
|
lockedZEN = 20
|
Vela settlement
/ \
success failure
| |
burn/clear unlock
```

A failed settlement must not permanently destroy the private balance.

9. Trigger contracts

Current Vela documentation describes trigger contracts capable of enqueueing follow-up requests after a normal request completes.

Noct should not introduce triggers unless they materially simplify a required settlement flow.

If used, the implementation must account for:

- callback ordering;
- isolated try/catch behavior;
- TRUSTPROCESS priority;
- termination conditions;
- replay/chaining risk.

Trigger-based architecture is therefore **optional and conditional**, not automatically part of V1.

10. Facilitator / relayer

A facilitator can submit requests and pay gas on behalf of users.

This is useful if Noct wants a gasless UX.

However:

```
facilitator != trusted custodian
```

The facilitator should not receive unnecessary private financial data.

Where Vela supports encrypted request payloads, Noct should preserve that privacy boundary.

The current Vela Facilitator demonstrates gas-relayed submission and TEE-confirmed settlement patterns.

11. Relayer failure model

A relayer/facilitator may:

- go offline;
- delay submission;
- submit duplicate requests;
- submit malformed requests;
- run out of gas.

Noct state must therefore rely on authenticated requests, nonces/replay protection and deterministic settlement rather than trusting the relayer.

12. Vela + zkVerify integration

Noct should conceptually use:

```
Noct private witness
↓
UltraHonk proof
↓
zkVerify
↓
verification result / aggregation receipt
↓
Noct transition authorization
↓
Vela WASM
↓
new private state
```

The exact ordering may differ for individual transitions after benchmarking.

The architecture MUST allow proof verification before state mutation.

13. State mutation rule

Never:

```
mutate state
then verify proof
```

Instead:

```
validate request
↓
verify authorization
↓
verify proof
↓
validate transition
↓
compute new state
↓
commit atomically
```

If any step fails, authoritative state remains unchanged.

14. Failure and rollback

Every state transition should be atomic from the protocol's perspective.

Required properties:

- no partial balance updates;
- no partial debt updates;
- no double withdrawal;
- no stale state commit;
- retry-safe processing;
- deterministic failure result;
- reconciliation metadata.

Use Vela's state-versioning/rollback capabilities where appropriate.

15. Vela local development

The current Starter Kit provides a Docker-based local environment with a local EVM chain and Vela components including Executor, Manager and Authority Service.

The local environment emulates the TEE; it is not equivalent to a production AWS Nitro Enclave.

Therefore:

> Passing local Vela tests does not by itself establish production TEE security.

Production deployment must separately validate attestation, deployment identity and enclave assumptions.

16. Vela source-of-truth policy

Implementation should prefer:

1. current repository source;
2. repository tests;
3. repository design docs;
4. current official Horizen docs;
5. older docs only when clearly compatible.

The Starter Kit and Nova repositories should be used as concrete implementation references.

17. References

- Starter Kit: <https://github.com/HorizenOfficial/vela-starterkit>
- Vela Nova: <https://github.com/HorizenOfficial/vela-nova>
- Vela core: <https://github.com/HorizenOfficial/vela>
- Vela Facilitator: <https://github.com/HorizenOfficial/vela-facilitator>

05-NOCT_IMPLEMENTATION_PLAN

Noct Finance — Implementation Plan

Version: 1.0-architecture-freeze

Status: Build plan for the implementation coding agent.

1. Principle

Build in vertical slices.

Do not implement the entire protocol first and test it later.

Every slice must include:

```
state
→ transition
→ proof/authorization
→ Vela execution
→ settlement
→ failure path
→ tests
→ performance measurement
```

2. Phase 0 — Repository and baseline

Tasks

- establish repository structure;
- pin toolchain versions;
- import/adapt Vela Starter Kit patterns;
- establish Noct WASM application;
- establish local Vela environment;
- establish Horizon testnet configuration;
- establish CI;
- establish test framework.

Acceptance

A minimal Noct WASM app can execute a deterministic request locally.

3. Phase 1 — Private account state

Implement:

```
AccountState
identityCommitment
accountNonce
cashUSDC
collateralUSDC
borrowedZEN
borrowedETH
debtZEN
debtETH
stateVersion
stateCommitment
pendingSettlement
```

Tests

- serialization;
- commitment stability;
- state-version increments;
- invalid state rejection;
- no negative balances.

4. Phase 2 — Deposit USDC

Implement:

```
DEPOSIT_USDC
```

Acceptance:

```
deposit 10
→ cashUSDC = 10
```

```
deposit 5
→ cashUSDC = 15
```

No supply should occur automatically.

Also test:

```
deposit
→ withdraw cash
```

without supplying.

5. Phase 3 — Supply USDC

Implement:

```
SUPPLY_USDC
```

Acceptance:

```
cashUSDC = 15
supply 13
→ cashUSDC = 2
→ collateralUSDC = 13
```

Test that the user cannot supply more than cash.

6. Phase 4 — Borrow

Implement:

```
BORROW_ZEN
BORROW_ETH
```

Borrowing must:

1. validate collateral;
2. validate risk parameters;
3. validate liquidity;
4. create debt;
5. credit private borrowed balance;
6. update state commitment;
7. preserve privacy.

Do not automatically withdraw the borrowed asset.

7. Phase 5 — Borrowed-asset withdrawal

Implement:

```
WITHDRAW_BORROWED_ASSET
```

Use the lock/settle/unlock model.

Test:

- success;
- failure;
- retry;
- duplicate request;
- stale request;
- insufficient borrowed balance.

8. Phase 6 — Repay

Implement:

```
REPAY_ZEN  
REPAY_ETH
```

Before coding overpayment behavior, freeze the exact policy.

Test:

- full repayment;
- partial repayment;
- zero repayment rejection;
- invalid asset;
- stale state;
- replay.

9. Phase 7 — Collateral release

Implement:

```
RELEASE_COLLATERAL
```

Required behavior:

```
collateralUSDC -= amount  
cashUSDC += amount
```

It must not directly send the released amount to the external wallet.

Test that risk constraints are respected after release.

10. Phase 8 — Cash withdrawal

Implement:

```
WITHDRAW_CASH_USDC
```

Required:

```
amount <= cashUSDC
```

Use lock/settle/unlock.

11. Phase 9 — Liquidation

Liquidation requires a dedicated implementation pass.

Before implementation freeze:

- oracle source;
- price freshness;
- liquidation threshold;
- liquidation bonus;
- close factor;
- liquidator settlement;
- private proof;
- partial liquidation;
- bad-debt handling.

Do not invent these parameters.

12. Phase 10 — ZK benchmark harness

This phase must occur before full ZK integration.

Benchmark:

witness generation
UltraHonk proving
zkVerify verification
aggregation
receipt availability
Vela transition
Horizon settlement

Record:

p50
p95
p99
max
CPU
RAM
proof size
circuit size
failure rate

13. Phase 11 — ZK circuits

Implement focused circuits:

Supply
Borrow
Repay
ReleaseCollateral
Withdraw
Liquidation

Each circuit gets:

- fixed version;
- fixed VK;
- fixed public-input schema;

- tests;
- negative tests;
- performance benchmark.

14. Phase 12 — zkVerify

Integrate:

```
UltraHonk
→ zkVerify
→ verification result
```

Then benchmark:

```
verification-only
vs
verification + aggregation receipt
```

Do not assume aggregation is faster or slower without measurements.

15. Phase 13 — Vela integration

Connect:

```
Noct client
→ request
→ Vela Manager
→ Executor
→ Noct WASM
→ state update
→ settlement
```

Test all failure/retry paths.

16. Phase 14 — Horizon integration

Implement and test:

- asset custody;
- application registration;
- Noct state commitments;
- proof/receipt references;
- external settlement;
- event reconciliation;
- deployment security.

Do not expose private position data in public contract storage/events.

17. Phase 15 — End-to-end test suite

Minimum scenarios:

Deposit

```
10 USDC → cash = 10
5 USDC → cash = 15
```

Deposit then withdraw without supply

```
15 cash
withdraw 2
→ cash = 13
```

Deposit → supply

```
15 cash
supply 13
→ cash = 2
→ collateral = 13
```

Borrow

```
collateral = 100
borrow ZEN
→ debt increases
→ borrowedZEN increases
```

Borrow then withdraw

```
borrowedZEN = 20
withdraw 20
→ borrowedZEN = 0
→ debt unchanged
```

Release collateral

```
cash = 5
collateral = 100
release 20
→ cash = 25
→ collateral = 80
```

Failed settlement

Private balance must recover.

Replay

Same transition cannot execute twice.

Stale state

Old state root cannot authorize a new transition.

18. Phase 16 — Security testing

Required categories:

- authorization bypass;
- replay;
- double spend;
- stale state;
- malformed proof;
- wrong VK;
- wrong application ID;
- wrong asset;
- wrong chain;
- invalid state root;
- overflow/underflow;
- precision/rounding;
- oracle manipulation;

- liquidation edge cases;
- settlement failure;
- facilitator abuse;
- denial of service.

19. Phase 17 — Performance testing

Run realistic workloads.

At minimum:

```
1 user
10 users
100 users
1,000 synthetic transitions
```

Measure:

- throughput;
- proof generation;
- verification;
- Vela processing;
- Horizon confirmation;
- end-to-end latency;
- concurrent prover capacity.

The protocol must not be declared production-ready solely because unit tests pass.

20. Phase 18 — Testnet readiness

Checklist:

- all frozen transitions implemented;
- no OPEN decision accidentally encoded;
- ZK toolchain pinned;
- VK hashes pinned;
- Vela app identity pinned;
- contracts audited internally;
- state migration strategy defined;
- failure/recovery tested;
- benchmark report attached;
- privacy review complete;
- monitoring enabled.

21. Implementation agent stop conditions

The coding agent **MUST** stop and flag an architecture issue if:

1. a required protocol rule is missing;
2. two architecture documents conflict;

3. a state transition requires an undefined field;
 4. a public input would reveal a private position;
 5. Vela cannot support the assumed operation;
 6. zkVerify cannot support the assumed proof;
 7. a circuit exceeds supported limits;
 8. an operation cannot satisfy the security invariant;
 9. external settlement can double-spend private state;
 10. a performance bottleneck invalidates the approved critical path.
- The agent MUST NOT solve these by silently inventing protocol rules.
-

22. Final build order

```
STATE
↓
DEPOSIT
↓
SUPPLY
↓
BORROW
↓
BORROWED-ASSET WITHDRAWAL
↓
REPAY
↓
COLLATERAL RELEASE
↓
CASH WITHDRAWAL
↓
LIQUIDATION
↓
ZK BENCHMARK
↓
ZK CIRCUITS
↓
zkVerify
↓
Vela integration
↓
Horizon settlement
↓
Security
↓
Performance
↓
Testnet
```

Do not reorder the critical security dependencies merely to make UI development faster.